R V College of Engineering
Department of Computer Science and Engineering
CIE - III(Improvement): Question Paper

Subject : (Code)	**Database Management Systems (CD252IA)**	**Semester :**	**5TH BE**
Date :/01/2025	**Duration :** 120 minutes	**Staff :Dr.HR/Dr.CNS/Dr.PD/Dr.SB/Dr.SNM/Dr.PH/Dr.PT/Dr.MNV**	
Name : Ananth	**USN :** 1RV22AI004	**Section :** AIML	**A/B/C/D/CD/CY/ISE/AIML**

S.N	PART-A	M	BT	Co
1.	What is the difference between lossless and lossy decomposition in DBMS?	2	L2	3
2.	List the two conditions for checking the Binary decomposition?	2	L1	2
3.	Define the Condition of 3NF?	2	L1	2
4.	Define a Transaction with example.	2	L1	1
5.	Elaborate and Define ACID properties	2	L1	1
	PART-B			
1a	Discuss the condition for two functional dependencies to be equivalent? Check whether relation R(A,B,C,D) having two FD sets FD1 = {A->B, B->C, AB->D} and FD2 = {A->B, B->C, A->C, A->D} are equivalent or not ?	5	L3	2
1b	Explain any 5 reasons for failure of transaction.	5	L2	4
2a	Explain the steps for finding Minimal Cover for Functional Dependencies. For the given set of FDs {A->C, AC->D, E->H, E->AD} find the minimal cover.	6	L3	2
2b	Write the algorithm for Testing whether a schedule is serializable or not.	4	L2	4
3a	Explain the properties of Attribute preservation and dependency preservation?	5	L2	3
3b	Given a relational schema R = { SSN, ENAME, PNUMBER, PNAME, PLOCATION, HOURS } and the decomposed table R1 = { ENAME, PLOCATION } and R2 = { SSN, PNUMBER, HOURS, PNAME, PLOCATION } and FD = { SSN → ENAME, PNUMBER → { PNAME, PLOCATION}, { SSN, PNUMBER } → HOURS }. Identify whether the given decomposition of R, R1 and R2 is lossless or lossy decomposition?	5	L3	4
4a	Given a relation R(A, B, C, D) and Functional Dependency set FD = { AB → CD, B → C }, determine whether the given R is in 2NF? If not convert it into 2 NF.	5	L3	1
b	With a transition diagram explain the states for transaction execution.	5	L2	2
5.	List and explain with examples the types of problems that can be encountered if two transactions are executing concurrently.	10	L2	1

Course Outcomes: After completing the course, the students will be able to:	
CO1	Understand and explore the needs and concepts of relational, NoSQL database and Distributed Architecture
CO2	Apply the knowledge of logical database design principles to real time issues.
CO3	Analyze and design data base systems using relational, NoSQL and Big Data concepts
CO4	Develop applications using relational and NoSQL database
CO5	Demonstrate database applications using various technologies.

BT-Blooms Taxonomy, CO-Course Outcomes, M-Marks

Marks Distribution	Particulars	CO1	CO2	CO3	CO4	CO5	L1	L2	L3	L4	L5	L6
	Test	19	20	7	14		8	31	21	-	-	-

Database Management Systems - CIE 3 Test (January 2025)

1. Difference between Lossless and Lossy Decomposition

Decomposition is the process of breaking a relation R into smaller relations $R_1, R_2, \ldots, R_n$ to remove redundancy (normalization).

Feature	Lossless (Non-Lossy) Join Decomposition	Lossy (Lossy Join) Decomposition
Definition	A decomposition is lossless if the natural join of the decomposed relations results in the **exact original relation** R.	A decomposition is lossy if the natural join of the decomposed relations results in a superset of the original relation (i.e., it creates **spurious tuples**).
Equation	$R_1 \bowtie R_2 = R$	$R_1 \bowtie R_2 \supset R$ (Contains extra, invalid data)
Information	No information is lost; the precise original data is preserved.	Information is "lost" in the sense that we cannot distinguish the original valid tuples from the fake (spurious) ones created by the join.
Desirability	**Required** for valid database design.	**Unacceptable** in database design.

2. Conditions for Checking Binary Decomposition (Lossless Join Test)

For a decomposition of a relation R into two relations R_1 and R_2 to be lossless, the following conditions must hold:

1. **Attribute Preservation:** The union of attributes of R_1 and R_2 must equal the attributes of R ($R_1 \cup R_2 = R$).
2. **Common Attribute Key Property:** The intersection of attributes ($R_1 \cap R_2$) must be a **Super Key** (determinant) for at least one of the decomposed relations.

Mathematically: The decomposition is lossless if:

$$(R_1 \cap R_2) \rightarrow R_1$$

OR

$$(R_1 \cap R_2) \rightarrow R_2$$

In simple terms: The common column(s) must be a unique key in either table 1 or table 2.

3. Condition of Third Normal Form (3NF)

A relation schema R is in **Third Normal Form (3NF)** if:

1. It is already in **Second Normal Form (2NF)**.
2. It contains **no transitive dependencies** for non-prime attributes.

Formal Definition: For every non-trivial functional dependency $X \rightarrow A$ that holds in R, one of the following conditions must be true:

- X is a **Super Key** of R.
- **OR**
- A is a **Prime Attribute** (part of some candidate key) of R.

This rule ensures that non-key attributes depend ONLY on the candidate key, and nothing else.

4. Definition of a Transaction with an Example

Definition: A **Transaction** is a logical unit of work that performs a sequence of operations (reads and writes) on a database. It must be treated as a single, indivisible unit: either all operations execute successfully, or none of them do.

Example: Money Transfer Consider transferring $500 from Account A to Account B. This involves two updates but is considered **one transaction**:

1. **Read(A)**: Check balance of A.
2. **A = A - 500**: Deduct amount.
3. **Write(A)**: Save new balance of A.
4. **Read(B)**: Get balance of B.
5. **B = B + 500**: Add amount.
6. **Write(B)**: Save new balance of B.
7. **Commit**: Make changes permanent.

If the system crashes after step 3, the transaction must be rolled back so money isn't lost.

5. ACID Properties

To ensure data integrity, every transaction must adhere to the **ACID** properties:

1. **Atomicity ("All or Nothing"):**
   - A transaction is an atomic unit of processing. It is either performed in its entirety or not performed at all.
   - *Example:* In a money transfer, if the debit happens but the credit fails, the debit must be undone (Rollback).
2. **Consistency (Correctness):**
   - A transaction must transform the database from one valid (consistent) state to another valid state. It must satisfy all integrity constraints (like keys, constraints, triggers).
   - *Example:* The sum of balances in Account A and B must be the same before and after the transfer.

3. **Isolation (Independence):**
    - Transactions executing concurrently should not interfere with each other. The intermediate state of a transaction should not be visible to other transactions until it is committed.
    - *Example:* If T1 is transferring money, T2 should not read the balance of A until T1 is finished.
4. **Durability (Persistence):**
    - Once a transaction commits, its changes are permanent and must survive any subsequent system failures (power loss, crashes).
5. **Durability (Persistence):**
    - Once a transaction commits, its changes are permanent and must survive any subsequent system failures (power loss, crashes).
    - *Example:* Once the bank says "Transfer Successful," the money must remain transferred even if the server reboots immediately after.

Part B: Functional Dependencies & Transaction Management

1.a. Equivalence of Functional Dependencies

Condition for Equivalence: Two sets of Functional Dependencies, F and G, are equivalent ($F \equiv G$) if and only if:

1. F **covers** G **(**$G \subseteq F^+$**):** Every FD in G can be inferred from F.
2. G **covers** F **(**$F \subseteq G^+$**):** Every FD in F can be inferred from G.

Problem: Check equivalence for Relation $R(A, B, C, D)$.

- $FD1 = \{A \to B, B \to C, AB \to D\}$
- $FD2 = \{A \to B, B \to C, A \to C, A \to D\}$

Step 1: Check if $FD1 \subseteq FD2^+$ **(Can FD2 imply FD1?)**

- $A \to B$: Directly present in FD2. **(Holds)**
- $B \to C$: Directly present in FD2. **(Holds)**
- $AB \to D$: Check closure $(AB)^+$ using FD2.
    - Start: $\{A, B\}$
    - From $A \to D$ (in FD2): Add D.
    - Closure: $\{A, B, D\}$. Since D is present, $AB \to D$ holds. **(Holds)**

Step 2: Check if $FD2 \subseteq FD1^+$ **(Can FD1 imply FD2?)**

- $A \to B$: Directly present in FD1. **(Holds)**
- $B \to C$: Directly present in FD1. **(Holds)**
- $A \to C$: Check closure $(A)^+$ using FD1.

- Start: $\{A\}$
- From $A \to B$: Add B. Now $\{A, B\}$.
- From $B \to C$: Add C.
- Closure: $\{A, B, C\}$. Since C is present, $A \to C$ holds. **(Holds)**

- $A \to D$: Check closure $(A)^+$ using FD1.
  - Start: $\{A\}$
  - From $A \to B$: Add B. Now $\{A, B\}$.
  - From $AB \to D$: Since we have both A and B, Add D.
  - Closure: $\{A, B, D\}$. Since D is present, $A \to D$ holds. **(Holds)**

Conclusion: Since both sets cover each other ($FD1 \subseteq FD2^+$ and $FD2 \subseteq FD1^+$), the functional dependency sets **are Equivalent**.

1.b. Reasons for Transaction Failure

A transaction may fail to complete its execution for various reasons. Five common causes are:

1. **System Crash (Hardware/Software Failure):**
   - A hardware malfunction (like a power failure or memory failure) or a bug in the database software/OS causes the system to stop running. The contents of the volatile storage (RAM) are lost.
2. **Transaction or System Error (Logical Error):**
   - The transaction itself performs an operation that causes it to fail, such as integer overflow, division by zero, or invalid parameter values.
3. **Local Errors / Exception Conditions:**
   - An exception condition occurs that is not necessarily an error but requires the transaction to be cancelled.
   - *Example:* An "Insufficient Funds" condition detected during a withdrawal transaction.
4. **Concurrency Control Enforcement:**
   - The DBMS aborts a transaction to violate serializability constraints or to resolve a deadlock.
   - *Example:* Transaction T1 is chosen as a "victim" in a deadlock scenario and is rolled back.
5. **Disk Failure:**
6. **Disk Failure:**
   - A failure occurs in the read/write heads of the disk or a physical block corruption, causing loss of data on the non-volatile storage.

2.a. Minimal Cover for Functional Dependencies

Steps for finding Minimal Cover (Canonical Cover):

1. **Decompose RHS:** Rewrite all FDs such that every Right-Hand Side (RHS) has only a single attribute.
   - Example: $A \to BC$ becomes $A \to B$ and $A \to C$.
2. **Remove Extraneous Attributes (LHS Simplification):** For every FD $X \to A$, check if a proper subset of X is sufficient to determine A using the other dependencies.
   - Example: If we have $\{A \to B, AB \to C\}$, we check if $A \to C$ or $B \to C$ holds. Since $A \to B$, A yields AB, so $AB \to C$ simplifies to $A \to C$.
3. **Remove Redundant FDs:** Check if any FD can be inferred from the remaining set of FDs (using closure). If yes, remove it.

Problem: Find Minimal Cover for $F = \{A \to C, AC \to D, E \to H, E \to AD\}$

Step 1: Decompose RHS

- $A \to C$
- $AC \to D$
- $E \to H$
- $E \to A$ (from $E \to AD$)
- $E \to D$ (from $E \to AD$)
- **Current Set:** $\{A \to C, AC \to D, E \to H, E \to A, E \to D\}$

Step 2: Remove Extraneous Attributes from LHS

- Check $AC \to D$. Can we simplify AC?
- We have $A \to C$ in the set.
- If we have A, we automatically get C. Thus, A is sufficient to get AC.
- We can replace $AC \to D$ with $A \to D$.
- **Current Set:** $\{A \to C, A \to D, E \to H, E \to A, E \to D\}$

Step 3: Remove Redundant FDs

- Check $E \to D$. Can we derive D from E using the other rules?
  - From $E \to A$ and $A \to D$, by transitivity, we get $E \to D$.
  - Therefore, $E \to D$ is **redundant**. Remove it.
- Check $A \to D$. Can we derive D from A using others?
  - Closure of A without this rule is $\{A, C\}$. D is not there. Keep it.
- **Final Set:** $\{A \to C, A \to D, E \to H, E \to A\}$

Answer: The Minimal Cover is $\{A \to C, A \to D, E \to H, E \to A\}$ (or $\{A \to CD, E \to AH\}$).

2.b. Algorithm for Testing Serializability

To check if a schedule is **Conflict Serializable**, we use the **Precedence Graph (Serialization Graph)** algorithm.

Algorithm Steps:

1. **Create Nodes:** Create a node in the graph for each active transaction ($T_1, T_2, \ldots, T_n$) in the schedule.
2. **Draw Edges (Identify Conflicts):** For every pair of operations Op_i (from transaction T_i) and Op_j (from transaction T_j) where $i \neq j$:
   - Check if they access the **same data item**.
   - Check if **at least one** of them is a **WRITE** operation.
   - If Op_i occurs **before** Op_j in the schedule, draw a directed edge from $T_i \rightarrow T_j$.

   Types of Conflicts:
   - **Read-Write ($T_i \rightarrow T_j$):** T_i reads X before T_j writes X.
   - **Write-Read ($T_i \rightarrow T_j$):** T_i writes X before T_j reads X.
   - **Write-Write ($T_i \rightarrow T_j$):** T_i writes X before T_j writes X.
3. **Check for Cycles:**
   - Traverse the graph to check for cycles.
   - **If a Cycle exists:** The schedule is **NOT** conflict serializable.
4. **Check for Cycles:**
   - Traverse the graph to check for cycles.
   - **If a Cycle exists:** The schedule is **NOT** conflict serializable.
   - **If No Cycle exists:** The schedule **IS** conflict serializable.

3.a. Properties of Decomposition (Preservation)

When decomposing a relation schema R into sub-relations $R_1, R_2, \ldots, R_n$, two properties are desirable (and usually required) to ensure the design is valid.

1. Attribute Preservation:

- **Definition:** The decomposition must contain all the attributes present in the original relation. No data columns should be lost.
- **Condition:** $R_1 \cup R_2 \cup \cdots \cup R_n = R$.
- *Explanation:* If the original table has columns A, B, and C, the decomposed tables combined must still account for A, B, and C.

2. Dependency Preservation:

- **Definition:** The decomposition should allow us to enforce all the original Functional Dependencies (FDs) by checking only the individual decomposed relations, without having to join them back together.

- **Condition:** Let F be the set of FDs on R, and F_i be the set of FDs applicable to R_i. The decomposition is dependency preserving if $(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$.
- *Explanation:* If an FD $A \rightarrow B$ exists in the original system, ideally A and B should end up in the same table so the DBMS can enforce the constraint efficiently (e.g., using a Unique Key on A).

3.b. Lossless vs. Lossy Decomposition Problem

Given:

- **Relation R:** $\{SSN, ENAME, PNUMBER, PNAME, PLOCATION, HOURS\}$
- **Decomposition:**
  - $R_1 = \{ENAME, PLOCATION\}$
  - $R_2 = \{SSN, PNUMBER, HOURS, PNAME, PLOCATION\}$
- **FDs:**
  1. $SSN \rightarrow ENAME$
  2. $PNUMBER \rightarrow \{PNAME, PLOCATION\}$
  3. $\{SSN, PNUMBER\} \rightarrow HOURS$

Step 1: Find Intersection (Common Attributes)

$$R_1 \cap R_2 = \{PLOCATION\}$$

Step 2: Check if Intersection is a Super Key For a decomposition to be **Lossless**, the common attribute(s) must be a **Super Key** (determinant) for **at least one** of the decomposed relations.

- **Check for R_1:** Does $PLOCATION \rightarrow R_1$?
  - i.e., Does $PLOCATION \rightarrow \{ENAME\}$?
  - Looking at the FDs, $PLOCATION$ is not on the Left-Hand Side of any FD. It does not determine $ENAME$. **(False)**
- **Check for R_2:** Does $PLOCATION \rightarrow R_2$?
  - i.e., Does $PLOCATION \rightarrow \{SSN, PNUMBER, HOURS, PNAME\}$?
  - No FD supports this. **(False)**

Conclusion: Since the intersection $\{PLOCATION\}$ is **not a key** for either R_1 or R_2, the condition for lossless join fails.

Answer: The given decomposition is **Lossy (Lossy Join)**.

4.a. Normalization to 2NF

Given:

- **Relation:** $R(A, B, C, D)$
- **Functional Dependencies:** $F = \{AB \rightarrow CD, B \rightarrow C\}$

Step 1: Identify Candidate Key

- To find the key, we check the closure of attributes.
- Closure of $\{A, B\}$:
  - Start: $\{A, B\}$
  - Apply $B \rightarrow C$: $\{A, B, C\}$
  - Apply $AB \rightarrow CD$: $\{A, B, C, D\}$
- Since $\{A, B\}$ determines all attributes, AB **is the Candidate Key**.
- **Prime Attributes:** A, B
- **Non-Prime Attributes:** C, D

Step 2: Check for 2NF

- **Definition:** A relation is in 2NF if it is in 1NF and contains **no partial dependencies**. A partial dependency occurs when a non-prime attribute depends on a *proper subset* of a candidate key.
- **Analysis:**
  - $AB \rightarrow CD$: This is a full dependency (AB is the full key).
  - $B \rightarrow C$: Here, C (non-prime) depends on B, which is a **proper subset** of the key AB.
- **Conclusion:** The relation is **NOT in 2NF** due to the partial dependency $B \rightarrow C$.

Step 3: Convert to 2NF To normalize, we decompose the relation to remove the partial dependency. We place the determinant (B) and its dependent (C) in a separate table.

1. **Relation R_1:** Created from the partial dependency $B \rightarrow C$.
   - Attributes: $\{B, C\}$
   - Key: B
2. **Relation R_2:** Contains the remaining attributes and the key.
   - Attributes: $\{A, B, D\}$ (C is removed, but B stays as the link/foreign key).
   - Key: AB
   - FDs: $AB \rightarrow D$.

Final 2NF Decomposition:

- $R_1(B, C)$
- $R_2(A, B, D)$

4.b. Transaction State Diagram

A transaction must be in one of the following states during its execution lifecycle.

State Transition Diagram:

```
stateDiagram-v2
    [*] --> Active : Begin Transaction
    Active --> PartiallyCommitted : End of Transaction (Read/Write done)
    Active --> Failed : Error / Crash
    PartiallyCommitted --> Committed : Successful Check
    PartiallyCommitted --> Failed : System Failure / Integrity Violation
    Failed --> Aborted : Rollback
    Aborted --> Terminated
    Committed --> Terminated
    Terminated --> [*]
```

Explanation of States:

1. **Active:** The initial state where the transaction enters and executes its read/write operations.
2. **Partially Committed:** Entered after the last statement of the transaction has been executed. It waits here while the DBMS ensures all changes can be permanently written to disk.
3. **Failed:** Entered if a normal execution cannot proceed (due to hardware failure, logical error, or concurrency check failure). It can be reached from Active or Partially Committed states.
4. **Aborted:** After a transaction fails, it is rolled back (changes undone) to restore the database to the state before the transaction started.
5. **Committed:** If the checks in the Partially Committed state succeed, the transaction commits, meaning changes are permanently saved (Durable).
6. **Terminated:** The transaction leaves the system.

5. Concurrency Problems (Anomalies)

When multiple transactions execute concurrently in an uncontrolled manner, several problems (anomalies) can occur that violate database consistency and isolation.

1. The Lost Update Problem

Definition: This occurs when two transactions read the same data item, modify it, and write it back. The update performed by the first transaction is overwritten by the second transaction, effectively "losing" the first update.

Example: Consider an account with Balance $X = 1000$.

- **T1** wants to withdraw 100.
- **T2** wants to withdraw 200.

Time	Transaction T1	Transaction T2	Value of X

1	`Read(X)` (Reads 1000)		1000
2		`Read(X)` (Reads 1000)	1000
3	`X = X - 100` (Local: 900)		1000
4		`X = X - 200` (Local: 800)	1000
5	`Write(X)` (Writes 900)		**900**
6		`Write(X)` (Writes 800)	**800**

Result: The final balance is 800. It should be $1000 - 100 - 200 = 700$. T1's withdrawal is lost.

2. The Dirty Read Problem (Temporary Update)

Definition: This occurs when a transaction reads a data item that has been modified by another transaction that has **not yet committed**. If the other transaction fails and rolls back, the first transaction has read invalid (dirty) data.

Example: Consider $X = 1000$. T1 updates X but fails later.

Time	Transaction T1	Transaction T2	Value of X
1	`Read(X)`		1000
2	`Write(X = 2000)`		**2000**
3		`Read(X)` (Reads 2000)	2000
4		`Write(X = 2000 + 100)`	**2100**
5	**FAIL & ROLLBACK**		1000

Result: T1 is rolled back, so X should be 1000. However, T2 has already read the uncommitted value (2000) and used it. T2 is working with data that never legally existed.

3. The Unrepeatable Read Problem (Inconsistent Analysis)

Definition: This occurs when a transaction reads the same data item twice and gets different values because another transaction modified the item between the two reads.

Example: Consider $X = 1000$. T1 reads X twice.

Time	Transaction T1	Transaction T2	Value of X
1	`Read(X)` (Reads 1000)		1000

2		`Read(X)`	1000
3		`Write(X = 500)`	**500**
4		**Commit**	500
5	`Read(X)` (Reads 500)		500

Result: T1 read X=1000 at step 1 and X=500 at step 5. This inconsistency can cause logic errors in reports or calculations requiring a stable snapshot.

4. The Phantom Read Problem

Definition: This occurs when a transaction executes a query that retrieves a set of rows matching a condition. Another transaction then inserts or deletes a row that matches that condition. If the first transaction repeats the query, it sees a different set of rows (a "phantom" row appears or disappears).

Example:

- **T1:** `SELECT * FROM Employee WHERE Salary > 50000` (Returns 3 rows).